

```
In [ ]: # import the code in the module and make it available to use
# "pd" or "np" act as a namespace that keeps all the attributes of the module together.
# Namespaces are useful for keeping your code readable and organized.
import pandas as pd
import numpy as np
```

Question 1

```
In [ ]: #
monthly = pd.read_csv('monthly2022.csv', index_col=0, header=None)
T = monthly.shape[0]
```

```
In [ ]: # getting the columns
basicstats = monthly.describe().transpose() #
basicstats = basicstats.drop(['min', 'max'], axis=1) #
```

```
In [ ]: #
basicstats=basicstats.apply(lambda x: x*100 if x.name!='count' else x)
basicstats
```

```
In [ ]: skew = monthly.skew().rename('skew')
zstat1 = (skew/np.sqrt(6/T)).rename('zstat1') #
kurtosis = monthly.kurtosis().rename('kurtosis')
zstat2 = (kurtosis/np.sqrt(24/T)).rename('zstat2') #
```

```
In [ ]: #
summ_stat=pd.concat([basicstats, skew, zstat1, kurtosis, zstat2], axis=1)
```

```
In [ ]: #
make_float = lambda x: x if x.name=='count' else round(x, 3)
summ_stat.apply(make_float)
```

```
In [ ]: #
jan_only = monthly.iloc[:,12]
T = jan_only.shape[0]
```

```
In [ ]: # getting the columns
basicstats = jan_only.describe().transpose()
basicstats = basicstats.drop('max', axis=1)
basicstats=basicstats.apply(lambda x: x*100 if x.name!='count' else x) #
```

```
In [ ]: skew = jan_only.skew().rename('skew')
zstat1 = (skew/np.sqrt(6/T)).rename('zstat1')
kurtosis = jan_only.kurtosis().rename('kurtosis')
zstat2 = (kurtosis/np.sqrt(24/T)).rename('zstat2')
```

```
In [ ]: # combining all calculations into one dataframe
pd.concat([basicstats, skew, zstat1, kurtosis, zstat2], axis=1)
```

Question 2

```
In [ ]: # delete all variables and functions in namespace
%reset
# Import the modules
import pandas as pd
```

```
import numpy as np
from datetime import datetime
from tqdm.notebook import tqdm #
import statsmodels.api as sm
```

```
In [ ]: # Define the function to calculate the fitted value from recursive OLS
def pred_from_OLS(X, Y, M):
    """Compute the next period predicted value based on recursive OLS regression.
    Args:
        X : The independant variables
        Y : The dependant variable
        M : Initial length of estimation window for out-of-sample tests
    Returns:
        Y_hat: The predicted value based on expanding windows
    """
    Y_Hat = np.full((len(Y),1), np.nan)
    Y = np.asarray(Y)
    X = np.asarray(X)
    X = sm.add_constant(X) #

    # We start the index at M+1 because the first element of predicted return is at t=M+2.
    for i in range(M+1,len(Y)):
        Y1 = Y[1:i]
        X1 = X[0:i-1,:]
        reg = sm.OLS(Y1, X1, missing='drop').fit()
        Y_Hat[i] = reg.predict(X[i-1,:]) #
    return Y_Hat
```

```
In [ ]: # Define the function to compute two out-of-sample measures of performance
def oos_eval(Y, Pred_Mean, Pred_Var, OOS_SSE_Hist):
    """This function computes OOS R^2 and CEV difference based on
    a vector of predicted mean and variance
    Args:
        Y: The excess returns
        Pred_Mean: The time series of predicted mean
        Pred_Var: The time series of predicted variance
        OOS_SSE_Hist: Out-of-sample sum of squares of forecast error
                     based on historical mean
    Returns:
        OOS_R2: Out-of-sample R^2
        CE: certainty equivalence of the out-of-sample return
    """
    OOS_SSE = np.sum((Y[M+1:]-Pred_Mean[M+1:])**2) #
    OOS_R2 = 1-OOS_SSE/OOS_SSE_Hist #
    w0 = ((1/gam)*(Pred_Mean/Pred_Var)).clip(None,1.5) #
    r0 = Y*w0
    CE = np.mean(r0[M+1:]-gam/2*np.var(r0[M+1:],ddof=1) #
    return OOS_R2, CE
```

```
In [ ]: # Create the table for in-sample R^2, out-of-sample R^2 and certainty equivlance
def GW2008(X, Y, Y_Hat, M):
    """Compute the in-sample and out-of-sample predictability
    statistics required to reproduce the plots from Goyal & Welch (2008).
    Args:
        X: The independent variables
        Y : The dependant variable
        Y_Hat: The vector of predicted values based on expanding windows
        M : Initial length of estimation window for out-of-sample tests
    Returns:
        data: a dataframe of the statistics of in-sample and out-of-sample tests
    """
    # Create the benchmark using historical average
    Hist_Mean = np.asarray(Y.expanding().mean().shift()) #
    Hist_Variance = np.asarray(Y.expanding().var().shift()) #
```

```

Y = np.asarray(Y)
X = np.asarray(X)
Y_Hat = np.asarray(Y_Hat)
X = sm.add_constant(X)

# I. In sample
Y_1 = Y[1:]
X_1 = X[0:len(Y)-1,:]
reg = sm.OLS(Y_1, X_1, missing='drop').fit()

# I.1 OLS Model (non truncated)
y1 = Y_1-np.mean(Y_1)
e1 = Y_1-reg.predict(X_1)
IS_R2 = 1-np.sum(e1**2)/np.sum(y1**2)

# I.2 OLS Model (truncated)
e1 = Y_1-reg.predict(X_1).clip(0, None)
IS_R2_T = 1-np.sum(e1**2)/np.sum(y1**2)

# II. Out of sample
# II.1 Historical mean model
OOS_SSE_Hist = np.sum((Y[M+1:]-Hist_Mean[M+1:])**2)
# Benchmark Certainty Equivalence
w0 = ((1/gam)*(Hist_Mean/Hist_Variance)).clip(None, 1.5);
r0 = Y*w0
CE0 = np.mean(r0[M+1:])-gam/2*np.var(r0[M+1:], ddof=1)

# II.2 Forecast based on regression with expanding window
OOS_R2, CE1 = oos_eval(Y, Y_Hat, Hist_Variance, OOS_SSE_Hist) #

# II.3 Truncated forecast based on regression with expanding window
OOS_R2_T, CE2 = oos_eval(Y, Y_Hat.clip(0, None), Hist_Variance, OOS_SSE_Hist)

#
data = pd.DataFrame({'$R_{IS}^2$:IS_R2','$R^2_{IS}$ (Truncated)':IS_R2_T,
                    '$R^2_{OS}$':OOS_R2,
                    '$R^2_{OS}$ (Truncated)':OOS_R2_T,'$\Delta$ CEV$:CE1-CE0,
                    '$\Delta$ CEV$ (Truncated)': CE2-CE0},index=[indep_var])*100

return data

```

```

In [ ]: def Pool_Avg(Y, Y_Hat, M):
        """Compute the out-of-sample predictability statistics based on pooling average.
        Args:
            Y : The dependant variable
            Y_Hat: The vector of predicted values based on expanding windows
            M : Initial length of estimation window for out-of-sample tests
        Returns:
            data: a data frame of the statistics of the out-of-sample tests
        """
        # Create the benchmark using historical average
        Hist_Mean = np.asarray(Y.expanding().mean().shift())
        Hist_Variance = np.asarray(Y.expanding().var().shift())
        Y = np.asarray(Y)
        Y_Hat = np.asarray(Y_Hat)

        # Benchmark (Historical Average)
        OOS_SSE_Hist = np.sum((Y[M+1:]-Hist_Mean[M+1:])**2)
        # Benchmark Certainty Equivalence
        w0 = ((1/gam)*(Hist_Mean/Hist_Variance)).clip(None, 1.5);
        r0 = Y*w0
        CE0 = np.mean(r0[M+1:])-gam/2*np.var(r0[M+1:], ddof=1)

        # Preapre for the pooling average
        Y_Hat_Mean = np.mean(Y_Hat,axis=1)
        Y_Hat_Median = np.full((len(Y)), np.nan)

```



```

    for i in range(M+1,len(Y)):
        Y_Hat_Median[i] = np.median(Y_Hat[i,:])
    # Alternative: Y_Hat_Median = np.median(Y_Hat,axis=1), this gives a warning message
    OOS_Error_OLS_Pool_Mean = Y-Y_Hat_Mean
    OOS_Error_OLS_Pool_Median = Y-Y_Hat_Median

    # Combination forecast based on mean of various forecasts
    OOS_R2_Pool_Mean, CE_Pool_Mean = oos_eval(Y, Y_Hat_Mean,
                                              Hist_Variance, OOS_SSE_Hist)

    # Combination forecast based on median of various forecasts
    OOS_R2_Pool_Median, CE_Pool_Median = oos_eval(Y, Y_Hat_Median,
                                                  Hist_Variance, OOS_SSE_Hist)

    #
    data = pd.DataFrame({'$R_{OS}^2$ (Pooled Mean)':OOS_R2_Pool_Mean,
                        '$\Delta$ CEV$ (Pooled Mean)':CE_Pool_Mean-CE0,
                        '$R_{OS}^2$ (Pooled Median)':OOS_R2_Pool_Median,
                        '$\Delta$ CEV$ (Pooled Median)':CE_Pool_Median-CE0},
                        index=['Combination'])*100
    return data

```

```

In [ ]: # Import Data
df_monthly = pd.read_excel('PredictorData2022.xlsx',sheet_name="Monthly")
# Parse the dates properly
time = [str(d) for d in df_monthly.yyyymm]
df_monthly.index = pd.to_datetime(time,format="%Y%m") #

```

```

In [ ]: # Variable Construction
df_monthly['ExRet'] = df_monthly['CRSP_SPvw']-df_monthly['Rfree']
df_monthly['DP'] = np.log(df_monthly['D12'])-np.log(df_monthly['Index'])
df_monthly['DY'] = np.log(df_monthly['D12'])-np.log(df_monthly['Index'].shift())
df_monthly['EP'] = np.log(df_monthly['E12'])-np.log(df_monthly['Index'])
df_monthly['DE'] = np.log(df_monthly['D12'])-np.log(df_monthly['E12'])
df_monthly['tms'] = df_monthly['lty']-df_monthly['tbl']
df_monthly['dfr'] = df_monthly['corpr']-df_monthly['ltr']
df_monthly['dfy'] = df_monthly['AAA']-df_monthly['BAA']

#
df_monthly['infl'] = df_monthly['infl'].shift().copy()

# Construction of dependent and independent variables
dep_var = 'ExRet'
indep_vars = ['DE','svar','dfr','lty','ltr','infl','tms',
              'tbl','dfy','DP','DY','EP','b/m','ntis']

#
subperiod = df_monthly.index>='1926-12-01'
df = df_monthly[subperiod]
M = 240 #
gam = 3 #

```

```

In [ ]: #
df_pred = pd.DataFrame(pred_from_OLS(df[[x for x in indep_vars
                                         if x not in('DE','tms')]],df[dep_var], M))
df_pred.columns = ['All']

#
for indep_var in tqdm(indep_vars):
    df_pred[indep_var] = pred_from_OLS(df[indep_var], df[dep_var], M)

```

```

In [ ]: # Create the Table for the in- and out-of-sample performance
pd.options.display.float_format = '{:,.3f}'.format

```

```
# Get rid of d/e and tms to avoid perfect multicollinearity
data = GW2008(df[[x for x in indep_vars if x not in ('DE', 'tms')]],
              df[dep_var], df_pred['All'], M)
data.rename(index={data.index[0]: 'All'}, inplace=True)

#
for indep_var in indep_vars:
    x = GW2008(df[indep_var], df[dep_var], df_pred[indep_var], M)
    data = pd.concat([data, x])
data
```

```
In [ ]: # Create table for combination forecasts
Pool_Avg(df[dep_var], df_pred[indep_vars], M)
```

```
In [ ]:
```